2.13 Ошибки. Создание и обработка

error - интерфейс, определенный в пакете `builtin`

```
type error
    interface {
    Error()
    string
}
```

Создание ошибок

```
package main

import
"fmt"
import
"errors"

func main() {
    err := errors.New("something
    went wrong")
    fmt.Println(err.Error())
    err = fmt.Errorf("Another
    error")
    fmt.Println(err.Error())
}
```

MERION

Пользовательские типы ошибок

```
package main

import "fmt"


type MyError struct {
    Code int
    Message string
}

func (e *MyError) Error() string {
    return fmt.Sprintf("Error %d: %s", e.Code, e.Message)
}

func main() {
    err := &MyError{Code: 404, Message: "Not Found"}
    println(err.Error())
}
```

MERION

```
result, err := someFunction()
if err != nil {
    println("Возникла ошибка", err.Error())
}
```

Добавление контекста к ошибкам при их распространении

```
func doSomething() error {
    err := someFunctionThatMightFail()
    if err != nil {
        return fmt.Errorf("doSomething failed: %w", err)
    }
    return nil
}
```

MERION

Развертывание ошибок

```
if err != nil {
    if wrappedErr := errors.Unwrap(err);
    wrappedErr != nil {
        // handle the wrapped error
    }
}
```

MERION

Отложенная обработка ошибок

```
func readFile(filename string) ([]byte, error) {
    file, err := os.Open(filename)
    if err != nil {
        return nil, err
    }
    defer file.Close()

    data, err := ioutil.ReadAll(file)
    if err != nil {
        return nil, err
    }
    return data, nil
}
```

Определение пользовательского типа ошибки

```
type MyError struct {
    Code int
    Message string
}

func (e *MyError) Error() string {
    return fmt.Sprintf("Error %d: %s", e.Code, e.Message)
}
```

MERION

errors.As

```
package main

import fmt

func someFunction() error {
    originalErr := &MyError{Code: 404, Message: "Not Found"}
    return fmt.Errorf("ошибка выполнения операции: %w", originalErr)
}

func main() {
    err := someFunction()
    if err != nil {
        var myErr *MyError
        if errors.As(err, &myErr) {
            fmt.Printf("Получена ошибка MyError: Code=%d, Message=%s\n", myErr.Code,
myErr.Message)
        } else {
            fmt.Println("Получена другая ошибка:", err)
        }
    }
}
```

MERION

Зачем использовать errors.As?

- **Извлечение данных из ошибки**: Если ошибка содержит полезную информацию (например, код ошибки или контекст), errors.As позволяет извлечь её.

- **Работа с цепочкой обёрток**: Ошибки в Go часто оборачиваются несколько раз (например, через разные уровни вызовов функций). errors.As помогает найти и извлечь ошибку нужного типа из этой цепочки.

errors.Is

errors.Is проверяет, соответствует ли ошибка определённому типу

```
package main

import (

    "errors"
    "fmt"
    "os"
)

func main() {
    _, err := os.Open("nonexistent.txt")
    if err != nil {
        if errors.Is(err, os.ErrNotExist) {
            fmt.Println("File does not exist")
        } else {
            fmt.Println("Other error:", err)
        }
    }
}
```

Лучшие практики

- **Всегда проверяйте ошибки**: Убедитесь, что каждое вызов функции, которая возвращает ошибку, проверяется.

- **Ясные сообщения об ошибках**: Предоставляйте информативные сообщения об ошибках для ясности.